

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА «ИНФОКОММУНИКАЦИОННЫЕ ТЕХНОЛОГИИ И СИСТЕМЫ СВЯЗИ»

КУРСОВАЯ РАБОТА (ПРОЕКТ)
ЗАЩИЩЕНА С ОЦЕНКОЙ

РУКОВОДИТЕЛЬ

ассистент	19.12.2025	А. Н. Головенков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ (ПРОЕКТУ)

По теме «ЦИКЛ ОТРИЦАТЕЛЬНОЙ СТОИМОСТИ»

по дисциплине: «Основы программирования»

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	2455	19.12.2025	А.С. Баишев
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

ОГЛАВЛЕНИЕ

1. ВВЕДЕНИЕ	3
2. ПОСТАНОВКА ЗАДАЧИ	4
3. АЛГОРИТМ	6
3.1. Описание структуры алгоритма	7
3.2. Алгоритм релаксации ребер relaxEdges	7
3.3. Алгоритм поиска отрицательного цикла findNegativeCycle	8
3.4. Алгоритм восстановления цикла restoreCycle	10
3.5. Пример пошагового выполнения	12
4. ИНСТРУКЦИЯ ПОЛЬЗОВАТЕЛЯ	16
5. ТЕСТОВЫЕ ПРИМЕРЫ	19
6. ЗАКЛЮЧЕНИЕ	24
7. СПИСОК ЛИТЕРАТУРЫ	25

1. ВВЕДЕНИЕ

Взвешенный ориентированный граф может содержать циклы отрицательной стоимости (отрицательного суммарного веса), при которых длина пути по этому циклу уменьшается с каждым обходом. В присутствии такого цикла в графе не существует корректного понятия «кратчайшего пути» (существуют сколь угодно короткие пути). Поэтому важно определить, есть ли в графе отрицательный цикл. Для этого удобно использовать модификацию алгоритма Беллмана-Форда, который за время $O(|V| \cdot |E|)$ позволяет обнаружить наличие отрицательного цикла. В данной работе формулируется задача поиска цикла отрицательной стоимости: при его наличии требуется вывести вершины этого цикла, а при отсутствии - сообщить об этом.

2. ПОСТАНОВКА ЗАДАЧИ

Задачей курсовой работы является разработка программы, определяющей наличие цикла отрицательной стоимости в ориентированном графе. В случае обнаружения такого цикла программа должна восстановить и вывести последовательность вершин, образующих данный цикл. Если отрицательный цикл в графе отсутствует, программа должна корректно сообщить об этом.

Входные данные программы задаются в виде текстового файла, содержащего описание графа: количество вершин, количество рёбер и список ориентированных рёбер с их весами. Результатом работы программы является текстовый файл с информацией о наличии или отсутствии отрицательного цикла, а также файл в формате DOT для визуализации графа с использованием системы Graphviz. Для решения задачи используется алгоритм Беллмана-Форда.

Пример А. Лучший случай выполнения алгоритма (отрицательный цикл отсутствует)

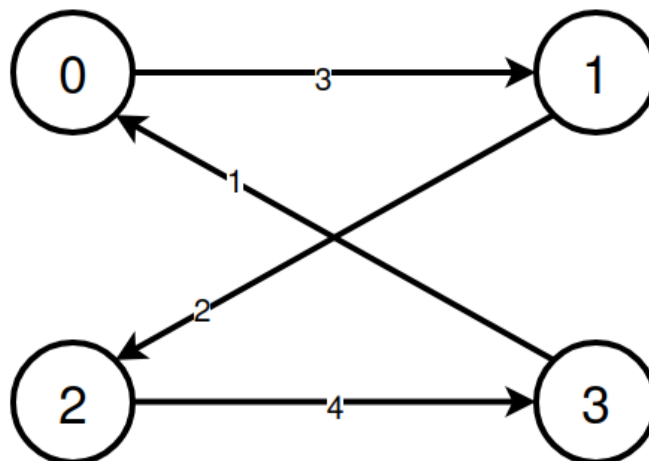


Рисунок 1. Граф без отрицательного цикла

Данный ориентированный граф (*Рисунок 1*) состоит из четырёх вершин и четырёх рёбер. Все рёбра имеют положительные веса. В графе присутствует цикл, проходящий через все вершины, однако суммарная стоимость данного цикла равна:

$$3 + 2 + 4 + 1 = 10 > 0$$

Следовательно, цикл отрицательной стоимости в данном графе отсутствует.

На ранних итерациях алгоритма Беллмана-Форда происходит стабилизация значений расстояний, и на очередной итерации не выполняется ни одной релаксации рёбер. Это позволяет завершить работу алгоритма без выполнения полного числа итераций. Данный случай является лучшим по времени выполнения

Пример Б: Худший случай выполнения алгоритма (отрицательный цикл присутствует)

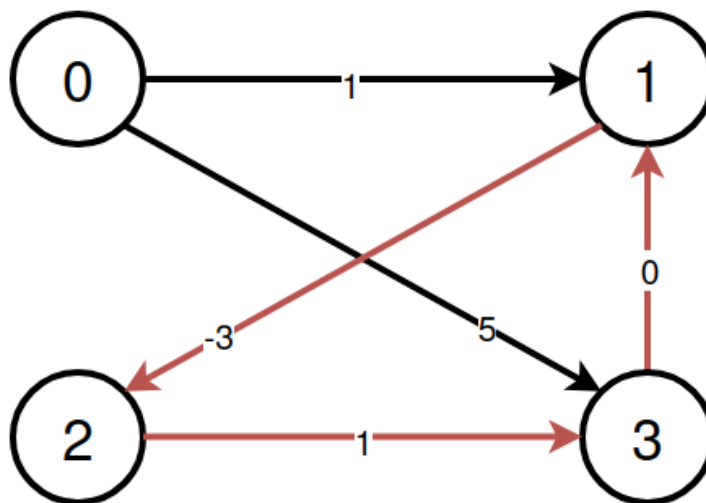


Рисунок 2. Граф с присутствующим отрицательным циклом

Данный ориентированный граф (*Рисунок 2*) также состоит из четырёх вершин, однако содержит рёбра с отрицательными весами. Вершины 1, 2 и 3 образуют цикл:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 1$$

Суммарная стоимость данного цикла равна:

$$-3 + 1 + 0 = -2 < 0$$

Таким образом, в графе присутствует цикл отрицательной стоимости.

В процессе работы алгоритма Беллмана-Форда релаксации рёбер происходят на каждой итерации, и отрицательный цикл обнаруживается только на последней итерации алгоритма. После этого выполняется дополнительный этап восстановления цикла по массиву предшественников. Данный случай является худшим по времени выполнения алгоритма.

3. АЛГОРИТМ

3.1. Описание структуры алгоритма

Ориентированный взвешенный граф представляется в виде списка ориентированных рёбер. Каждое ребро характеризуется начальной вершиной, конечной вершиной и весом. Для решения задачи поиска цикла отрицательной стоимости используется модифицированный алгоритм Беллмана-Форда.

Алгоритм основан на последовательной релаксации всех рёбер графа. В отличие от классического варианта, расстояния до всех вершин инициализируются нулевыми значениями, что позволяет обнаружить отрицательный цикл в любой компоненте связности графа. Если на последней итерации релаксации происходит улучшение хотя бы одного расстояния, это означает наличие цикла отрицательной стоимости. В этом случае выполняется восстановление цикла по массиву предшественников.

3.2. Алгоритм релаксации ребер **relaxEdges**

Описание:

Данный алгоритм выполняет одну итерацию релаксации всех рёбер графа. Для каждого ребра проверяется возможность улучшения расстояния до конечной вершины. В случае успешной релаксации обновляется массив расстояний и массив предшественников

Сложность:

Проход по всем рёбрам: $O(m)$

Псевдокод:

```
Procedure relaxEdges() :  
    updatedVertex = -1  
    for each edge (u, v, w) :  
        if dist[u] + w < dist[v] :  
            dist[v] = dist[u] + w  
            parent[v] = u
```

```

    updatedVertex = v
    return updatedVertex

```

Блок-схема:

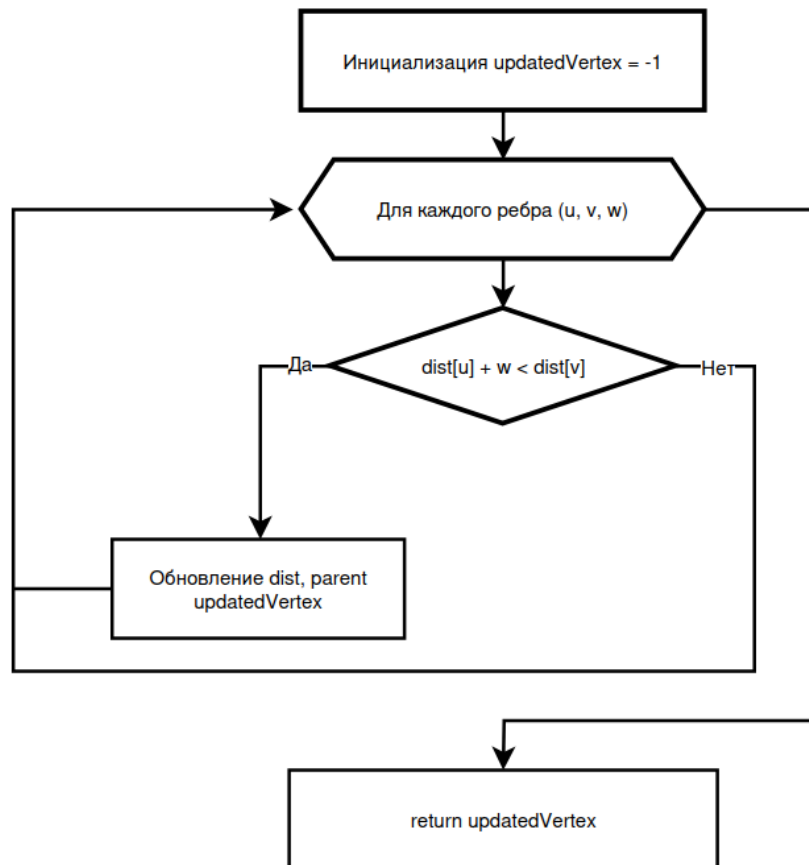


Рисунок 3. Блок-схема алгоритма релаксации ребер

3.3. Алгоритм поиска отрицательного цикла **findNegativeCycle**

Описание:

Алгоритм выполняет n итераций релаксации рёбер. Если на последней итерации произошло обновление расстояний, фиксируется вершина, принадлежащая отрицательному циклу. В противном случае делается вывод об отсутствии отрицательных циклов в графе.

Сложность: $O(n \cdot m)$.

Псевдокод:


```

Procedure findNegativeCycle():
    initialize dist[v] = 0 for all v
    initialize parent[v] = -1
    x = -1
    for i = 1 to n:
        x = relaxEdges()
    if x == -1:
        return false
    else:
        cycleStart = x
        return true

```

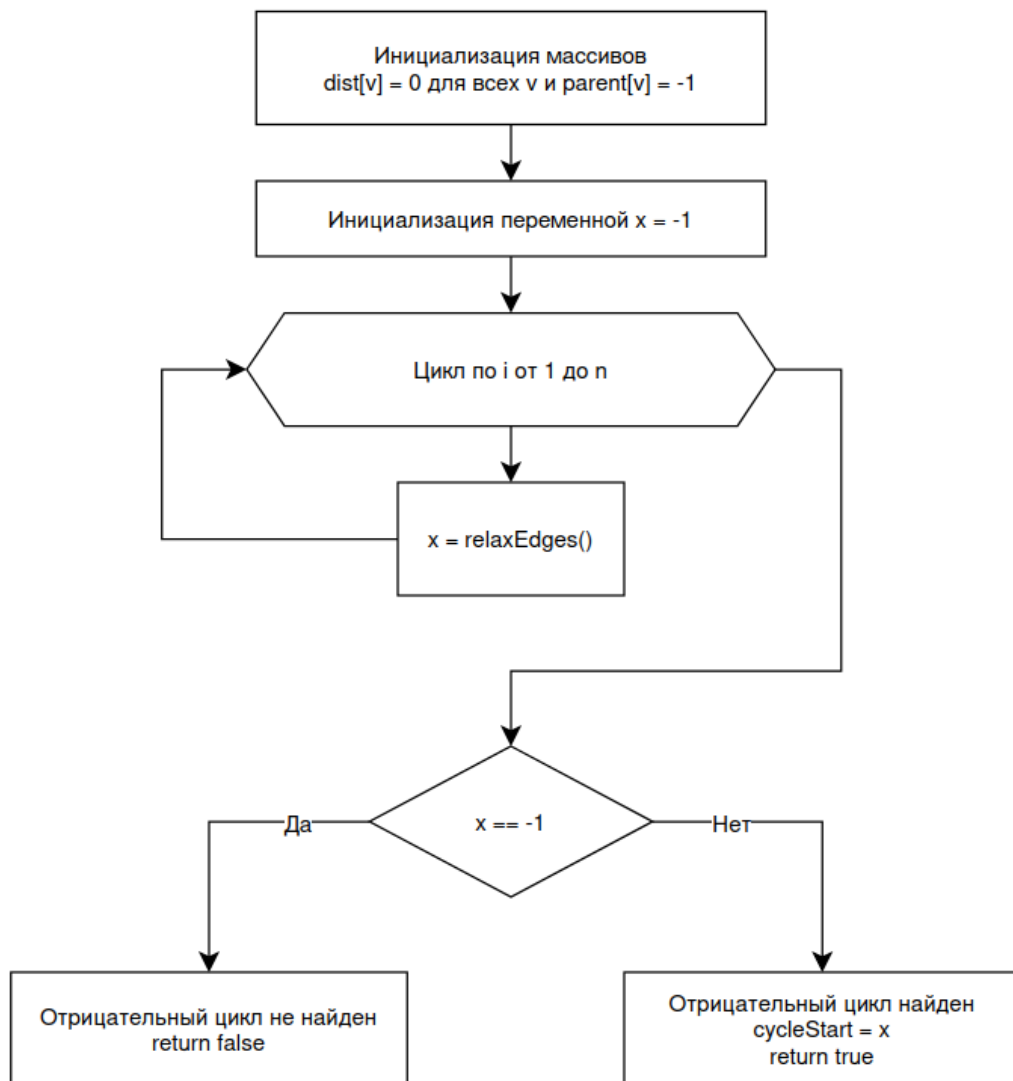


Рисунок 4. Блок-схема поиска отрицательного цикла

3.4. Алгоритм восстановления цикла `restoreCycle`

Описание:

Алгоритм восстанавливает цикл отрицательной стоимости по массиву предшественников. Для гарантированного попадания внутрь цикла выполняется n переходов по предшественникам, после чего осуществляется обход цикла до возврата в начальную вершину

Сложность:

Переходы по предшественникам: $O(n)$

Итоговая сложность: $O(n)$

Псевдокод:

```
Procedure restoreCycle(start):  
    v = start  
    for i = 1 to n:  
        v = parent[v]  
    cycle = empty list  
    cur = v  
    do:  
        cycle.add(cur)  
        cur = parent[cur]  
    while cur != v  
    reverse(cycle)  
    return cycle
```

Блок-схема:

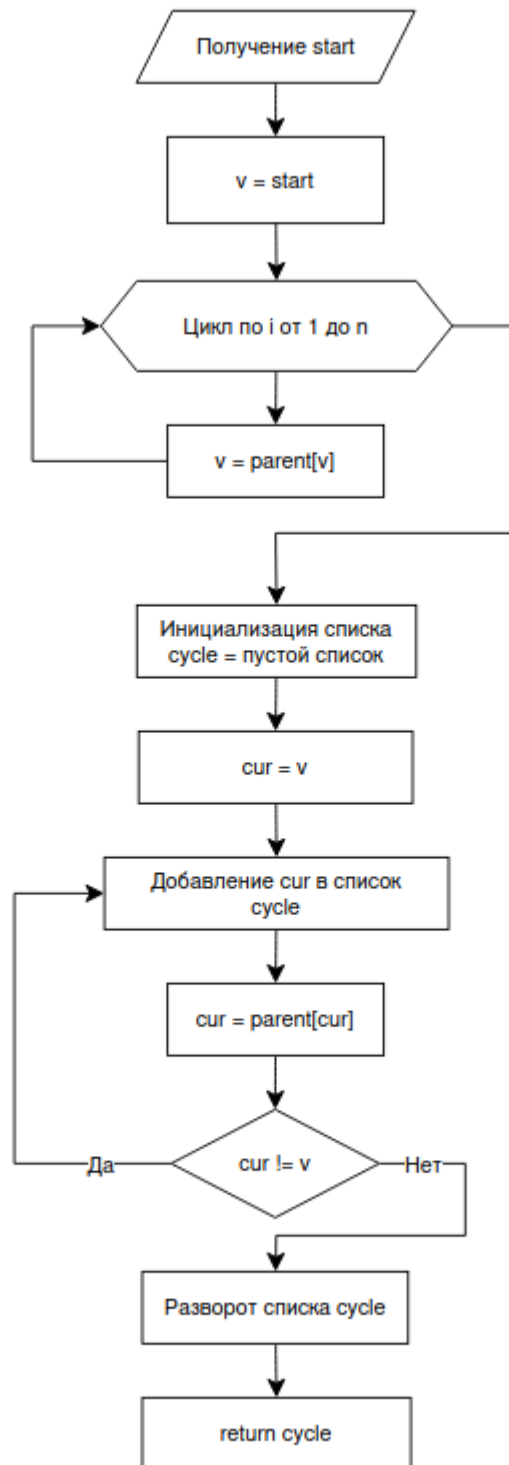


Рисунок 5. Блок-схема восстановления цикла

3.5. Пример пошагового выполнения

Шаг 1 - Инициализация алгоритма

На данном этапе выполняется инициализация массивов расстояний и предшественников. Для всех вершин графа расстояния устанавливаются равными нулю, а предшественники - неопределёнными.

```
dist:    [0, 0, 0, 0]
```

```
parent:  [-1, -1, -1, -1]
```

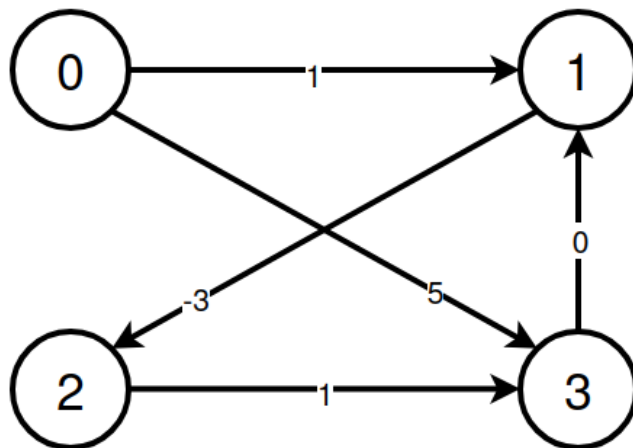


Рисунок 6. Пример исходного графа.

Шаг 2 - Первая итерация релаксации рёбер

Выполняется проход по всем рёбрам графа и проверка условия релаксации.

Результат первой итерации:

```
dist:    [0, -2, -3, -2]
```

```
parent:  [-1, 3, 1, 2]
```

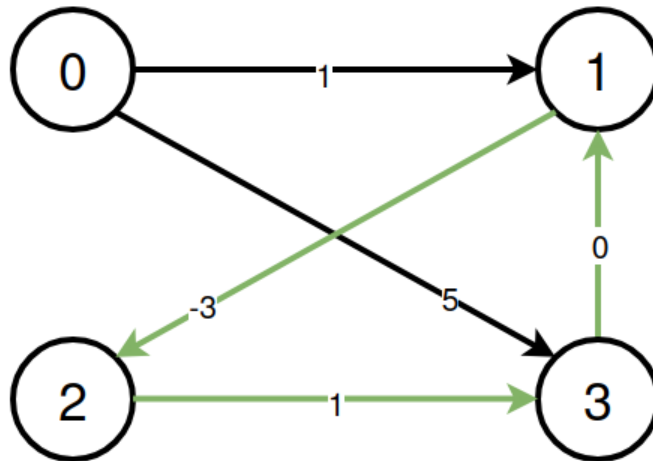


Рисунок 7. Граф с обозначенными ребрами, для которых выполнилось условие релаксации

Шаг 3 - Последующие итерации алгоритма

Алгоритм выполняет последующие итерации релаксации рёбер. На каждой итерации продолжают происходить улучшения расстояний, что указывает на наличие цикла отрицательной стоимости.

На последней итерации было зафиксировано улучшение для вершины 1, которая сохраняется как `cycleStart`.

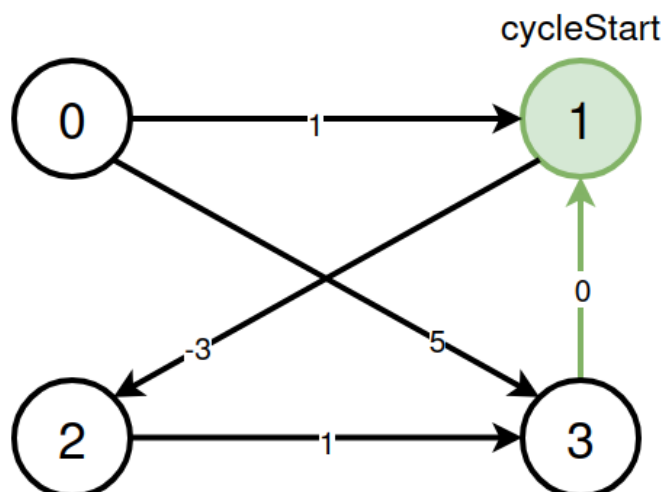


Рисунок 8. Граф с обозначенной вершиной `cycleStart` и ребром, по которому для нее произошло улучшение

Шаг 4 - Вход внутрь отрицательного цикла

Для гарантированного попадания внутрь отрицательного цикла выполняется последовательный переход по массиву предшественников n раз.

$v = \text{cycleStart} = 1$

$v \rightarrow \text{parent}[v] \rightarrow \text{parent}[\text{parent}[v]] \rightarrow \dots$

В результате данного перехода получена вершина, принадлежащая циклу отрицательной стоимости.

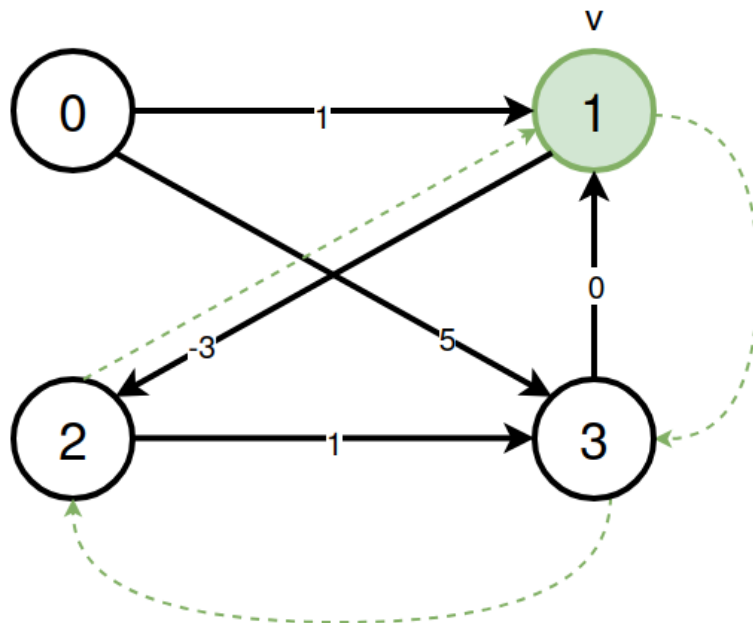


Рисунок 9. Результат перехода по массиву предшественников на графе

Шаг 5 - Восстановление цикла отрицательной стоимости

После попадания внутрь цикла выполняется обход по массиву предшественников до возврата в исходную вершину.

Полученный цикл:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

Для корректного порядка вершин выполняется разворот списка.

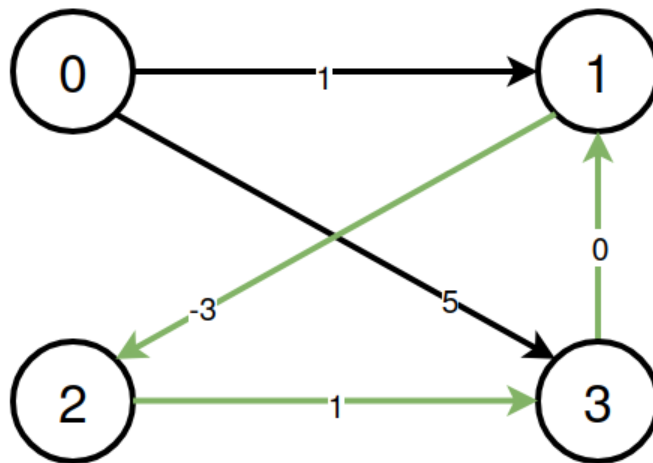


Рисунок 10. Граф с обозначенным на нем найденным на нем отрицательным
ЦИКЛОМ

3.6. Используемые структуры данных

Struct Edge:

```
int from
int to
int weight
```

Class Graph:

```
int vertexCount
vector<Edge> edges
```

Class NegativeCycleFinder:

```
vector<int> dist
vector<int> parent
Graph graph
int cycleStart
```

4. ИНСТРУКЦИЯ ПОЛЬЗОВАТЕЛЯ

Для работы с программой необходимо подготовить текстовый файл с описанием графа. В первой строке файла указываются два числа: количество вершин N и количество рёбер M . Далее в каждой из M строк указываются три числа: $u\ v\ w$, где u - начальная вершина ребра, v - конечная вершина, w - вес ребра. Вершины нумеруются от 0 до $N-1$. Вес ребра может быть положительным или отрицательным. Запуск программы осуществляется из терминала командой:

```
g++ -std=c++17 main.cpp graph.cpp  
negative_cycle.cpp -o negcycle
```

```
./negcycle input.txt
```

где `input.txt` - имя файла с описанием графа.

После запуска программа создаёт два файла:

- `cycle.txt` - текстовый файл с информацией о наличии отрицательного цикла.
 - Если цикл найден, в файле содержится последовательность вершин, образующих цикл отрицательной стоимости.
 - Если цикла нет, выводится сообщение: Цикл отрицательной стоимости не найден
- `cycle.dot` - файл в формате DOT для визуализации графа через Graphviz.
 - Рёбра, входящие в найденный цикл, выделяются цветом.
 - Для построения изображения используйте команду: `dot -Tpng cycle.dot -o cycle.png`

Для просмотра содержимого созданных файлов, необходимо использовать следующие команды:

```
cat cycle.txt
```



```
cat cycle.dot
```

Пример вывода:

```
chel@chel-BMH-WDX9:~$ cat cycle.txt
Найден отрицательный цикл:
1 2 3 1
chel@chel-BMH-WDX9:~$ cat cycle.dot
digraph G {
    0 -> 1 [label="1"];
    1 -> 2 [label="-3", color=red];
    2 -> 3 [label="1", color=red];
    3 -> 1 [label="0", color=red];
    0 -> 3 [label="5"];
}
```

Рисунок 11. Вывод в терминал

После, пользователь вводит следующую команду:

```
open cycle.png
```

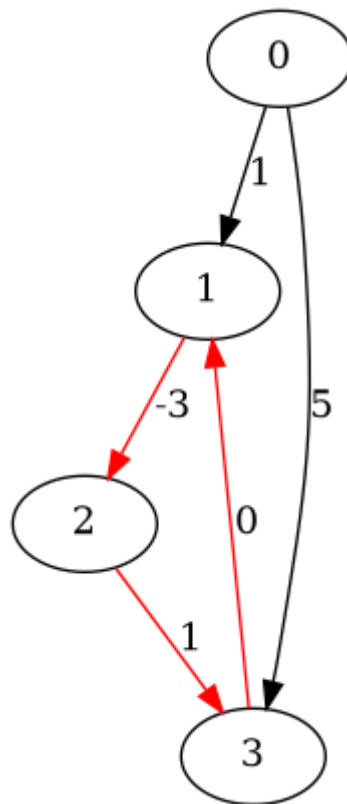


Рисунок 12. Пример визуализации

5. ТЕСТОВЫЕ ПРИМЕРЫ

Тест 1

input.txt

4 5

0 1 1

1 2 -3

2 3 1

3 1 0

0 3 5

cycle.txt

Найден отрицательный цикл:

3 1 2 3

cycle.dot

```
digraph G {  
    0 -> 1 [label="1"];  
    1 -> 2 [label="-3", color=red];  
    2 -> 3 [label="1", color=red];  
    3 -> 1 [label="0", color=red];  
    0 -> 3 [label="5"];  
}
```

cycle.png

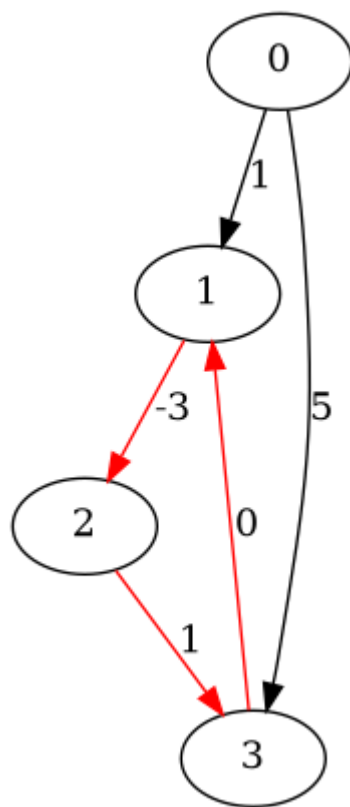


Рисунок 13. Файл cycle.png (тест 1)

Тест 2

input.txt

5 6

0 1 2

1 2 2

2 3 -6

3 1 1

2 4 3

4 0 1

cycle.txt

Найден цикл отрицательной стоимости:

1 2 3 1

cycle.dot

```
digraph G {  
    0 -> 1 [label="4"];  
    1 -> 2 [label="1", color=red];  
    2 -> 3 [label="-6", color=red];  
    3 -> 1 [label="1", color=red];  
    3 -> 4 [label="2"];  
    4 -> 0 [label="1"];  
    2 -> 4 [label="3"];  
}
```

cycle.png

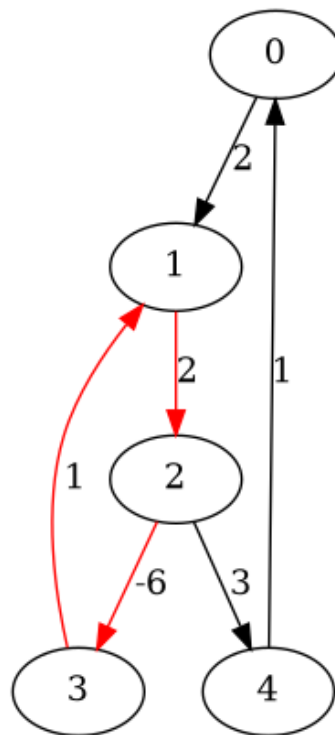


Рисунок 14. cycle.png (тест 2)

Тест 3

input.txt

3 4

0 1 2

1 2 3

2 0 1

0 2 4

cycle.txt

Цикл отрицательной стоимости не найден

cycle.dot

```
digraph G {  
    0 -> 1 [label="2"];  
    1 -> 2 [label="3"];  
    2 -> 0 [label="1"];  
    0 -> 2 [label="4"];  
}
```

cycle.png

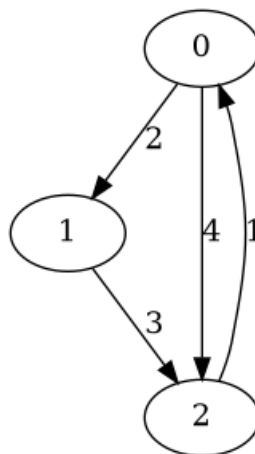


Рисунок 15. cycle.png (тест 3)

6. ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была рассмотрена задача обнаружения цикла отрицательной стоимости в ориентированном графе с целочисленными весами рёбер. Данная задача является важной в теории графов и имеет практическое применение в анализе сетей, оптимизации маршрутов и выявлении противоречивых зависимостей в различных прикладных областях.

Для решения поставленной задачи был выбран алгоритм Беллмана-Форда, позволяющий не только определить факт наличия отрицательного цикла, но и восстановить сам цикл при его существовании. Алгоритм был модифицирован таким образом, чтобы обеспечить корректное восстановление последовательности вершин, образующих цикл отрицательной стоимости.

Разработанный алгоритм включает разбор входных данных, выполнение алгоритма, восстановление отрицательного цикла и формирование выходных данных, включая файл визуализации в формате DOT для последующего отображения графа с помощью Graphviz.

7. СПИСОК ЛИТЕРАТУРЫ

1. Ахо, А. Построение и анализ вычислительных алгоритмов / А. Ахо, Дж. Хопкрофт, Дж. Ульман. - Москва : Мир, 1979. - 536 с. - Текст : электронный // <https://djvu.online/file/Slt3J8EdmWsy6> (дата обращения: 01.12.2025)

2. Рейнгольд, Э. Комбинаторные алгоритмы / Э. Рейнгольд, Ю. Нивергельт, Н. Део. - Москва : Мир, 1980. - Текст : электронный // <https://libcats.org/book/585922> (дата обращения: 01.12.2025)

3. Алгоритм Беллмана - Форда : статья // Википедия : свободная энциклопедия. - URL: https://ru.wikipedia.org/wiki/%D0%90%D0%BB%D0%B3%D0%BE%D1%80%D0%B8%D1%82%D0%BC_%D0%91%D0%B5%D0%BB%D0%BB%D0%BC%D0%B0%D0%BD%D0%B0_%E2%80%94%D0%A4%D0%BE%D1%80%D0%B4%D0%B0 (дата обращения: 01.12.2025) - Текст : электронный.